

Accounting Analytics with Python

Analytics Powered by Large Language Models (LLM)

Sharif Islam, DBA, CPA, CMA

2026-07-15

Table of contents

Preface	1
About the Author	3
Acknowledgement	5
I. Part I: Foundations	7
1. Introduction to Accounting Analytics	9
1.1. What Is Accounting Analytics?	9
1.2. Why Now? The Shift Toward Data-Driven Accounting	10
1.3. Accounting Analytics vs. Traditional Accounting and Auditing	10
1.4. The Four Types of Analytics	11
1.5. The Accounting Analytics Workflow	11
1.6. Where Accounting Analytics Shows Up in Practice	12
1.7. Careers and Certifications	12
1.8. A Note on Tools: Why Python and Quarto?	13
1.9. Chapter Summary	13
1.10. Discussion Questions	14
1.11. Exercises	14
2. Python and Quarto Environment Setup	15
2.1. Why This Chapter Matters	15
2.2. The Three Tools You Need	15
2.3. Step 1: Install Python	16
2.4. Step 2: Install Quarto	17
2.5. Step 3: Install a Code Editor	17
2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document	18
2.6.1. 1. A Plain Python Script (.py)	18
2.6.2. 2. A Jupyter Notebook (.ipynb)	19
2.6.3. 3. A Quarto Document (.qmd)	20
2.7. Your First Quarto Document	23
2.8. Rendering This Book’s Project (HTML and PDF)	25

2.9. Installing the Python Packages You'll Need	26
2.10. Organizing Your Project Folder	27
2.11. Troubleshooting Common Setup Issues	28
2.12. Chapter Summary	28
2.13. Exercises	29
3. Data Fundamentals for Accountants	31
3.1. Why Data Structure Matters Before You Start Coding	31
3.2. Tidy Data Principles	31
3.3. Levels of Measurement	33
3.4. Data Types You'll Encounter	34
3.4.1. A Special Note on Dates	34
3.5. Core Accounting Data Structures	34
3.5.1. Chart of Accounts (COA)	35
3.5.2. General Ledger (GL)	35
3.5.3. Subsidiary Ledgers	35
3.5.4. Trial Balance	35
3.6. Transaction-Level vs. Summarized Data	36
3.7. Common Data Quality Issues in Accounting Data	36
3.8. A Preview Look at a Real Dataset	37
3.9. Chapter Summary	38
3.10. Discussion Questions	38
3.11. Exercises	39
 II. Part II: Core Data Skills	 41
4. Data Wrangling with Pandas and Polars	43
4.1. What is Data Wrangling?	43
4.1.1. Data Wrangling Steps and Techniques	44
4.1.2. Data Wrangling vs. Data Cleaning	46
4.1.3. Importance of Data Wrangling	47
4.2. Two Tools, One Set of Concepts	47
4.3. Reading Data In	48
4.4. pandas	48
4.5. polars	49
4.6. pandas	49
4.7. polars	50
4.7.1. Reading with an Explicit Schema	50
4.8. pandas	50
4.9. polars	51

4.10. Selecting, Filtering, and Sorting	51
4.10.1. Filtering on a Single Condition	51
4.11. pandas	51
4.12. polars	52
4.12.1. Combining Multiple Conditions	52
4.13. pandas	52
4.14. polars	53
4.14.1. Sorting	53
4.15. pandas	54
4.16. polars	54
4.17. Cleaning Data	54
4.17.1. Step 1: Fix Currency Formatting	55
4.18. pandas	55
4.19. polars	56
4.19.1. Step 2: Standardize Text Fields	57
4.20. pandas	57
4.21. polars	57
4.21.1. Step 3: Fix Inconsistent Dates	58
4.22. pandas	58
4.23. polars	59
4.23.1. Step 4: Handle Missing Values	60
4.24. pandas	60
4.25. polars	61
4.25.1. Step 5: Check for Duplicates	61
4.26. pandas	62
4.27. polars	62
4.28. Merging and Joining Tables	62
4.29. pandas	62
4.30. polars	63
4.30.1. Inner Join	63
4.31. pandas	63
4.32. polars	64
4.32.1. Left Join	64
4.33. pandas	64
4.34. polars	65
4.35. Reshaping Data	65
4.35.1. Wide: Total Debits by Account and Department	65
4.36. pandas	65
4.37. polars	66
4.37.1. Back to Long (Tidy) Format	67
4.38. pandas	67

Table of contents

4.39. polars	67
4.40. Grouping and Aggregating	68
4.41. pandas	68
4.42. polars	69
4.42.1. Multi-Level Grouping	69
4.43. pandas	70
4.44. polars	70
4.45. Polars Lazy Frames: Wrangling Large Accounting Datasets	71
4.45.1. Eager vs. Lazy: The Same Pipeline, Two Ways	71
4.46. Chapter Summary	74
4.47. Discussion Questions	74
4.48. Exercises	75
 5. Exploratory Data Analysis for Financial Data	 77
 6. Data Visualization for Accounting	 79
 III. Part III: Applied Accounting Analytics	 81
 7. Financial Statement Analysis with Python	 83
 8. Audit Analytics	 85
 9. Fraud Detection and Anomaly Detection	 87
 10. Revenue Recognition and Transaction-Level Analysis	 89
 11. Tax Analytics	 91
 IV. Part IV: Predictive Methods and GenAI	 93
 12. Introduction to Predictive Modeling	 95
 13. Generative AI in Accounting	 97
 V. Part V: Communication and Capstone	 99
 14. Automated Reporting and Dashboards with Quarto	 101
 15. Data Ethics, Governance, and Reproducibility	 103

16. Capstone Project	105
References	107
Appendices	109
A. A Primer on Python	109
B. Financial Statement Basics	111

Preface

Welcome to *Accounting Analytics with Python*. This book is intended for accounting undergraduate students. However, students from other disciplines such as Finance are very likely to be benefited from using the book.

About the Author

Sharif Islam, DBA, CPA, CMA is an Associate professor in School of Accountancy in Southern Illinois University Carbondale (SIUC). He is a licensed CPA in Illinois and a Certified Management Accountant (CMA). He teaches Advanced Cost Accounting, Auditing, Accounting Information Systems, Machine Learning, and Analytics for Accounting Data. He earned his doctorate from Louisiana Tech University. His manuscripts are selected for “Best Research Paper Award” in several conferences of American Accounting Association (AAA). His research also got 2024 “Notable Contribution to the Literature Award” by AIS section of AAA. He published research in *Accounting Horizons*, *Journal of Accounting and Public Policy*, *Journal of Information Systems*, *Advances in Accounting*, *Journal of Emerging Technologies in Accounting*, *Issues in Accounting Education*, and *Managerial Auditing Journal*. His research interests lie at the intersection of Accounting and Data Science. More about his teaching and research can be found in his [personal website](#).

Acknowledgement

To prepare the book, I took help from many sources on the internet and published materials. Many of them are cited in the book. I acknowledge the contribution of all of those resources that help me to prepare the book for the students.

Part I.

Part I: Foundations

1. Introduction to Accounting Analytics

Learning Objectives

By the end of this chapter, you should be able to:

- Define accounting analytics and distinguish it from traditional accounting and auditing
- Explain why data analytics has become central to accounting practice
- Describe the four major types of analytics (descriptive, diagnostic, predictive, prescriptive) and give an accounting example of each
- Outline the typical accounting analytics workflow, from raw data to a decision
- Identify career paths and professional certifications related to accounting analytics

1.1. What Is Accounting Analytics?

Accounting has always been about data. Every transaction a business records — a sale, a payment, a depreciation entry — becomes a data point that eventually rolls up into financial statements. What has changed in the last two decades is not *that* accountants work with data, but the *volume*, *speed*, and *tools* available for working with it.

Accounting analytics is the application of data analysis techniques — statistical, visual, and computational — to accounting and financial data in order to support decisions in areas such as financial reporting, auditing, tax, and management accounting. It sits at the intersection of three fields:

- **Accounting knowledge** — understanding what the numbers mean (revenue recognition, internal controls, GAAP/IFRS rules)
- **Data skills** — the ability to collect, clean, and analyze data (which is where Python comes in)
- **Business judgment** — knowing what question is worth asking and what a result actually implies for a decision

A useful way to think about it: traditional accounting produces the numbers; accounting analytics asks *what the numbers are telling us* and *what we should do about it*.

1.2. Why Now? The Shift Toward Data-Driven Accounting

Several forces have pushed analytics from a “nice to have” to a core accounting skill:

1. **Data volume.** Modern ERP systems (SAP, Oracle, NetSuite) log every transaction electronically. A mid-sized company can generate millions of journal entries a year — far more than any auditor or accountant could review manually.
2. **Regulatory and audit expectations.** Standard-setters and regulators (including the PCAOB) increasingly expect auditors to test *entire populations* of transactions rather than small manual samples, which requires analytics tools.
3. **Automation of routine work.** Many manual bookkeeping and reconciliation tasks are increasingly automated, shifting accountants’ time toward analysis, interpretation, and communication of results.
4. **The rise of accessible tools.** Python, R, and business intelligence tools (Power BI, Tableau) have made analytics accessible to accountants without a computer science background — you don’t need to be a software engineer to benefit from these tools.
5. **CPA Exam evolution.** The AICPA has incorporated data analytics content into the CPA Exam Blueprint, particularly in the Business Analysis and Reporting (BAR) discipline, signaling that the profession considers this a core competency, not an elective skill.



Connect to Practice

Throughout this book, look for “Connect to Practice” boxes like this one, which tie a concept back to how it’s used in real accounting and audit work.

1.3. Accounting Analytics vs. Traditional Accounting and Auditing

It’s worth being precise about how this course differs from other accounting courses you’ve taken:

	Traditional Accounting/Auditing	Accounting Analytics
Primary tool	Spreadsheets, manual ledgers	Python, SQL, visualization tools
Sample size	Often a sample of transactions	Frequently the <i>entire population</i>
Focus	Was this recorded correctly?	What patterns, risks, or insights exist in the data?

	Traditional Accounting/Auditing	Accounting Analytics
Output	Journal entries, financial statements, audit opinions	Dashboards, models, flagged anomalies, predictions
Skillset emphasis	Rules and standards (GAAP/IFRS)	Rules and standards <i>plus</i> coding, statistics, visualization

This isn't a replacement for the accounting knowledge you've built in other courses — it's a set of tools that make that knowledge more powerful and scalable.

1.4. The Four Types of Analytics

A common framework in the analytics field breaks analysis into four types, moving from “what happened” to “what should we do.” Every technique in this book fits into one of these categories.

1. **Descriptive analytics** — *What happened?* Summarizing past data. Example: calculating total revenue by month, or the average days-sales-outstanding over the last year.
2. **Diagnostic analytics** — *Why did it happen?* Investigating causes and relationships. Example: determining why gross margin declined in Q3 by breaking revenue and cost data down by product line.
3. **Predictive analytics** — *What is likely to happen?* Using historical data to forecast. Example: predicting which customers are likely to become uncollectible accounts receivable.
4. **Prescriptive analytics** — *What should we do?* Recommending an action based on analysis. Example: recommending which vendors to flag for additional audit procedures based on a fraud-risk score.

Most introductory accounting analytics work — including much of this book — lives in the descriptive and diagnostic space, with an introduction to predictive methods later in the book (Chapter 12). Prescriptive analytics tends to require more advanced modeling and is largely beyond the scope of an undergraduate course, but it's useful to know where the field is heading.

1.5. The Accounting Analytics Workflow

Most accounting analytics projects — whether a homework assignment in this course or a real audit engagement — follow a similar sequence:

1. **Define the question.** What decision or risk are we trying to inform? (e.g., “Are there unusual journal entries near period-end that warrant investigation?”)

1. *Introduction to Accounting Analytics*

2. **Obtain the data.** Extract data from a source system — a general ledger export, a CSV file, a database query.
3. **Clean and prepare the data.** Real data is messy: missing values, inconsistent formats, duplicate entries. This step often takes the most time in practice.
4. **Explore and analyze.** Apply descriptive statistics, visualizations, or models to look for patterns.
5. **Interpret results in context.** A statistical pattern is not automatically meaningful — this is where accounting judgment matters most.
6. **Communicate findings.** Produce a report, dashboard, or memo that a non-technical stakeholder (a partner, a controller, a client) can act on.

This book is structured around this workflow. Chapters 2–3 build foundational tools and data literacy; Chapters 4–6 cover steps 2–4 in depth; Chapters 7–11 apply the workflow to specific accounting domains (financial statements, audit, fraud, revenue, tax); and later chapters extend into predictive methods, GenAI, and communicating results.

1.6. Where Accounting Analytics Shows Up in Practice

A few examples of accounting analytics in action:

- **External audit** — Using full-population testing and anomaly detection to identify unusual journal entries for further audit procedures, rather than testing a small manual sample.
- **Internal audit** — Continuous monitoring dashboards that flag control violations or policy exceptions as they occur, not just at year-end.
- **Financial planning & analysis (FP&A)** — Building forecasting models and variance-analysis dashboards to support budgeting decisions.
- **Forensic accounting** — Applying techniques like Benford’s Law (covered in Chapter 8) to detect potential financial statement fraud.
- **Tax** — Analyzing large transactional datasets to identify tax planning opportunities or compliance risks across jurisdictions.

1.7. Careers and Certifications

Accounting analytics skills are increasingly valued across public accounting, industry, and government. A few relevant credentials and pathways worth knowing about as you plan your career:

- **CPA (Certified Public Accountant)** — increasingly incorporates data analytics content, especially through the BAR discipline

- **CISA (Certified Information Systems Auditor)** — relevant for IT audit and control-focused roles
- **Data-analytics-specific certificates** offered by the AICPA and various universities

You don't need to become a data scientist to benefit from this course — the goal is *analytics literacy*: being comfortable enough with data tools that you can perform basic analyses yourself, and knowledgeable enough to critically evaluate analyses done by others (including, increasingly, by AI tools).

1.8. A Note on Tools: Why Python and Quarto?

This book uses **Python** as its programming language and **Quarto** as its authoring and reporting tool, for a few reasons that matter to your future career:

- Python is one of the most widely used languages in data analytics and is free and open-source, meaning you can continue using it after graduation without needing a paid license.
- The pandas library (introduced in Chapter 4) is purpose-built for exactly the kind of tabular, spreadsheet-like data accountants work with every day.
- Quarto allows you to combine narrative writing, code, and output (tables, charts) into a single reproducible document — which mirrors how analytics work is increasingly delivered in practice, as a reproducible report rather than a static spreadsheet.

You'll get hands-on with both tools starting in Chapter 2.

1.9. Chapter Summary

- Accounting analytics combines accounting knowledge, data skills, and business judgment to support decisions.
- The shift toward analytics is driven by data volume, regulatory expectations, automation, and accessible tools.
- Analytics can be descriptive, diagnostic, predictive, or prescriptive — most of this course focuses on the first two, with an introduction to the third.
- A consistent workflow (define, obtain, clean, analyze, interpret, communicate) underlies most accounting analytics projects, and structures the rest of this book.
- These skills map directly onto current and evolving areas of accounting practice, including the CPA Exam.

1.10. Discussion Questions

1. Choose one accounting role (e.g., external auditor, tax accountant, FP&A analyst). How might accounting analytics change the day-to-day work of someone in that role over the next five years?
2. Think of a business decision (not necessarily accounting-related) you or your family have made recently. Which of the four analytics types (descriptive, diagnostic, predictive, prescriptive) best describes the thinking that went into it?
3. Why do you think regulators like the PCAOB have pushed auditors toward analyzing full populations of transactions rather than samples? What are the potential benefits and risks of this shift?

1.11. Exercises

1. Find one recent (last 12 months) news article or company disclosure describing the use of data analytics or AI in an accounting, audit, or finance context. Write a short (150-word) summary of what the technology does and what problem it solves.
2. Look up the current AICPA CPA Exam Blueprint for the Business Analysis and Reporting (BAR) discipline. Identify two specific data-analytics-related skills listed and briefly explain, in your own words, what they mean.

2. Python and Quarto Environment Setup

Learning Objectives

By the end of this chapter, you should be able to:

- Install Python, Quarto, and a code editor on your own computer
- Explain the difference between a Jupyter notebook (`.ipynb`), a plain Python script (`.py`), and a Quarto document (`.qmd`)
- Create, edit, and render a Quarto document containing Python code
- Render a Quarto book to both HTML and PDF
- Install and import the Python packages used throughout this book (pandas, numpy, matplotlib)

2.1. Why This Chapter Matters

Every chapter after this one assumes you can do three things: write Python code, run it, and see the results in a nicely formatted document. This chapter gets that pipeline working on your own laptop, once, so that the rest of the course is about *accounting analytics* rather than *fighting with software installation*.

If you get stuck at any point in this chapter, don't worry — environment setup is almost always the hardest part of a data analytics course, and it only gets easier from here.

2.2. The Three Tools You Need

Tool	What it does	Analogy
Python	The programming language that does the actual data work	The engine

2. Python and Quarto Environment Setup

Tool	What it does	Analogy
Quarto	Combines your writing, Python code, and output into one document (HTML, PDF, Word)	The publishing system
A Code Editor (Also called Integrated Development Environment(IDE))	Where you write and run your code (this book uses Positron, but VS Code works identically)	The workshop

You need all three installed before you can complete the exercises in this book.

2.3. Step 1: Install Python

You can install the python from <https://www.python.org/downloads/>. We recommend you use this link to install a specific version of python. Your professor might recommend a specific version that you should use. If you use macOS, please click macOS to install python suitable for mac.

In addition, you can also install Python via the **Anaconda** or **Miniconda** distribution, because it comes bundled with the data packages we'll use (pandas, numpy, matplotlib) and makes managing versions easier than a bare Python install.

1. Download Miniconda from <https://docs.conda.io/en/latest/miniconda.html> (choose the installer for your operating system).
2. Run the installer with default settings.
3. Confirm the install worked by opening a terminal and typing:

```
python --version
```

You should see something like Python 3.11.x (the exact version doesn't need to match this).



Connect to Practice

If your future employer uses a specific data platform (e.g., a cloud data warehouse, or a company-managed Python environment), the *concepts* you learn here transfer directly — only the installation step changes.

2.4. Step 2: Install Quarto

1. Go to <https://quarto.org/docs/get-started/> and download the installer for your operating system.
2. Run the installer with default settings.
3. Confirm the install worked:

```
quarto --version
```

Since we'll render to PDF as well as HTML, also install a LaTeX distribution — Quarto includes a lightweight one you can install with a single command:

```
quarto install tinytex
```

This only needs to be done once per computer.

2.5. Step 3: Install a Code Editor

This book's examples use **Positron**, which can be downloaded from <https://positron.posit.co/download.html>. Positron is a code editor built specifically for data science that will feel very familiar if you've used VS Code before — the two share the same underlying editing engine, so keyboard shortcuts, extensions, and layout all work the same way. (If your instructor prefers plain VS Code, everything in this chapter applies identically.)

Figure 2.1 shows the general layout you'll be working in for the rest of this book: a file explorer on the left, an editor pane in the middle where you write your `.qmd` files, a terminal at the bottom where you run commands like `quarto render`, and a Source Control panel for Git (covered separately if your instructor uses version control in this course).

2. Python and Quarto Environment Setup

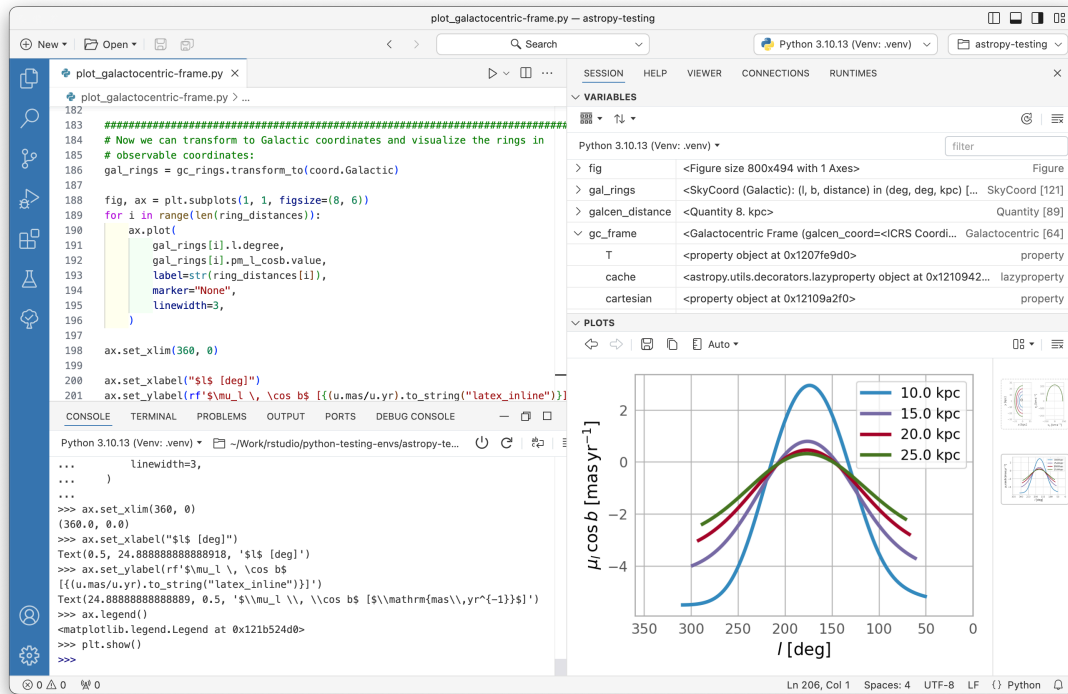


Figure 2.1.: A typical code-editor layout: file explorer, editor pane, session, and plots

Once installed, open your project folder in Positron/VS Code using **File → Open Folder**, and select the folder containing your `_quarto.yml` file.

2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

As you start doing accounting analytics work, you'll encounter three common file types for writing Python. It's worth understanding the difference early, since this book uses all three at different points.

2.6.1. 1. A Plain Python Script (.py)

A `.py` file contains only code — no narrative text, no formatted output. You run the whole file at once (or line by line) and see output printed to a terminal or console. This is the format used for production code, reusable functions, or automated jobs that don't need explanation baked in.

```
# analysis.py
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
print(df.head())
```

2.6.2. 2. A Jupyter Notebook (.ipynb)

A Jupyter notebook mixes code and narrative text in a sequence of **cells**, and shows output (tables, charts) directly underneath the code that produced it. This makes notebooks excellent for exploratory analysis — trying things out, looking at results immediately, and iterating.

Figure 2.2 shows a simplified notebook: a markdown cell explaining what's about to happen, a code cell that loads data, and the output (a data table) displayed directly below it.

2. Python and Quarto Environment Setup

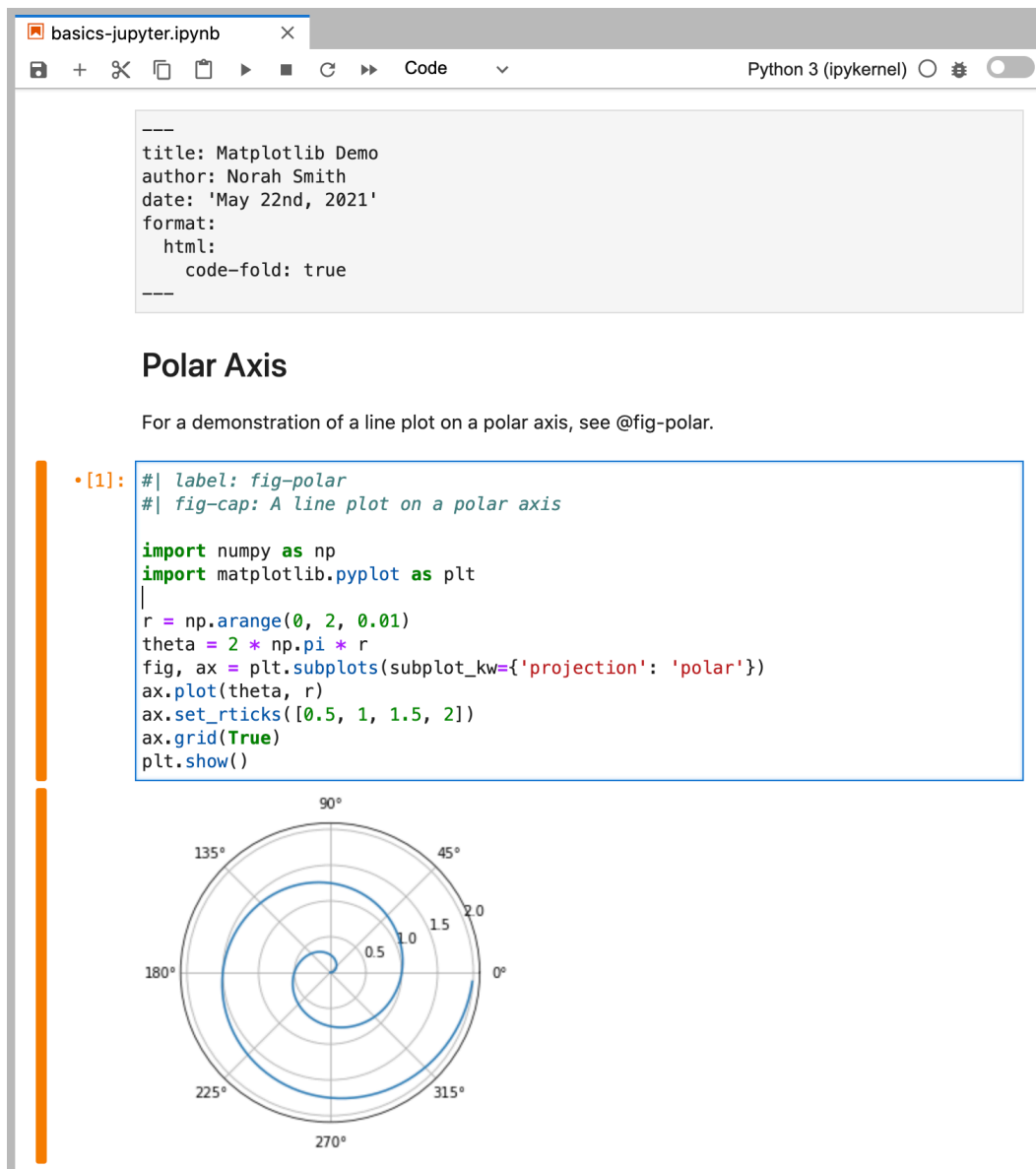


Figure 2.2.: A Jupyter notebook combines markdown cells, code cells, and their output in one scrollable document

You can open and run `.ipynb` files directly in Positron/VS Code, the same as in classic Jupyter.

2.6.3. 3. A Quarto Document (`.qmd`)

A Quarto document looks a lot like a notebook conceptually — narrative text plus embedded, runnable code — but it's a **plain text file** (easier to track with Git) that can render to multiple polished output formats: HTML, PDF, Word, and more. This is the format used for every chapter in this book.

A Quarto document has three parts:

1. A **YAML header** at the top (between `---` lines) with metadata like the title
2. **Narrative text** written in Markdown
3. **Code chunks**, marked with three backticks and curly braces

```
---
title: "Example Chapter"
---

## Analyzing the Trial Balance

Here we load the trial balance and preview the first few rows.

::: {.cell execution_count=1}
``` {.python .cell-code}
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
df.head()
```

::: {.cell-output .cell-output-display execution_count=1}
```{=html}
<div>
<style scoped>
 .dataframe tbody tr th:only-of-type {
 vertical-align: middle;
 }

 .dataframe tbody tr th {
 vertical-align: top;
 }

 .dataframe thead th {
 text-align: right;
 }
</style>
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>

```

## 2. Python and Quarto Environment Setup

```
<th>account</th>
<th>account_type</th>
<th>debit</th>
<th>credit</th>
</tr>
</thead>
<tbody>
<tr>
<th>0</th>
<td>Cash</td>
<td>Asset</td>
<td>12500</td>
<td>0</td>
</tr>
<tr>
<th>1</th>
<td>Accounts Receivable</td>
<td>Asset</td>
<td>8200</td>
<td>0</td>
</tr>
<tr>
<th>2</th>
<td>Inventory</td>
<td>Asset</td>
<td>15300</td>
<td>0</td>
</tr>
<tr>
<th>3</th>
<td>Prepaid Insurance</td>
<td>Asset</td>
<td>1200</td>
<td>0</td>
</tr>
<tr>
<th>4</th>
<td>Equipment</td>
<td>Asset</td>
<td>42000</td>
<td>0</td>
```

```

 </tr>
 </tbody>
</table>
</div>
```

:::
:::

```

The output above shows account balances before adjusting entries.

! Which format does this book use?

Every chapter in *this* book is written as a .qmd file, since we want a single, polished, professional document (in both HTML and PDF) at the end of the semester — not a folder of separate notebooks. However, your instructor may ask you to *practice and experiment* in a Jupyter notebook before copying finished code into your chapter or assignment .qmd file. Think of the notebook as your scratch paper and the .qmd file as your final submission.

2.7. Your First Quarto Document

Let's create a small Quarto document from scratch to make sure everything works end to end.

1. Create a new file called test.qmd in your project folder with the following content:

```

---
title: "My First Quarto Document"
format: html
---

## Testing My Setup

::: {.cell execution_count=2}
``` {.python .cell-code}
import pandas as pd

data = {"account": ["Cash", "Accounts Receivable", "Inventory"],
 "balance": [12500, 8200, 15300]}

df = pd.DataFrame(data)

```

## 2. Python and Quarto Environment Setup

```
df
```

::: {.cell-output .cell-output-display execution_count=2}
```${=html}
<div>
<style scoped>
 .dataframe tbody tr th:only-of-type {
 vertical-align: middle;
 }

 .dataframe tbody tr th {
 vertical-align: top;
 }

 .dataframe thead th {
 text-align: right;
 }
</style>
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>account</th>
 <th>balance</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>Cash</td>
 <td>12500</td>
 </tr>
 <tr>
 <th>1</th>
 <td>Accounts Receivable</td>
 <td>8200</td>
 </tr>
 <tr>
 <th>2</th>
 <td>Inventory</td>

```



```

 <td>15300</td>
 </tr>
 </tbody>
 </table>
</div>
...

:::
:::

```

If you can see a table above with three accounts, your Python and Quarto setup is

**2. Render it.** In the terminal, run:

```
quarto render test.qmd
```

**3. Open the result.** This creates `test.html` in the same folder — open it in your web browser and confirm you can see the table.

If this worked, your environment is fully set up. If you got an error, check the Troubleshooting section at the end of this chapter before moving on.

## 2.8. Rendering This Book's Project (HTML and PDF)

Your course textbook project is set up as a **Quarto book** (multiple chapters combined into one publication), which is slightly different from rendering a single file. From the terminal, inside your project folder (the one containing `_quarto.yml`):

```
quarto render
```

This single command renders *every* chapter listed in `_quarto.yml` and produces both formats we configured:

- `_book/index.html` — an HTML website you can open in a browser or host online
- a single PDF file inside `_book/` — a downloadable, printable version of the entire book

Figure 2.3 summarizes this process: your `.qmd` source files go in, `quarto render` executes the Python code and assembles the document, and both an HTML and a PDF version come out the other side from the *same source file* — you never have to write your content twice.

## 2. Python and Quarto Environment Setup

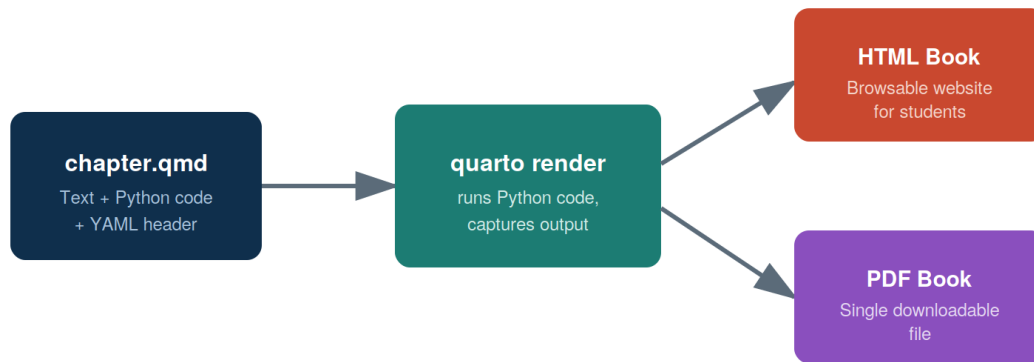


Figure 2.3.: The Quarto rendering workflow: one source document, two polished output formats

To preview your book live while editing (auto-refreshing as you save changes), use:

```
quarto preview
```

## 2.9. Installing the Python Packages You'll Need

This book relies on a small set of Python packages. If you installed Python via Anaconda/Miniconda, most of these are likely already available. To install (or confirm) them, run:

```
pip install pandas numpy matplotlib jupyter
```

Confirm they're installed correctly by running this in a `.qmd` code chunk or a notebook cell:

```
```{python}
import pandas as pd
import numpy as np
import matplotlib

print(pd.__version__)
print(np.__version__)
print(matplotlib.__version__)
```
```

If this runs without errors and prints three version numbers, you're ready for Chapter 3.

## 2.10. Organizing Your Project Folder

As your book (or your homework project) grows, keeping files organized will save you real time. Figure 2.4 shows the recommended structure, which matches how this book’s own project folder is organized: one `.qmd` file per chapter, a shared `images/` folder for figures, and the configuration file (`_quarto.yml`) at the top level.

```

accounting-analytics-book/
 _quarto.yml
 index.qmd
 01-introduction-to-accounting-analytics.qmd
 02-python-quarto-setup.qmd
 03-data-fundamentals.qmd
 ... (remaining chapter files)
 references.qmd
 references.bib
 appendix-a-python-primer.qmd
 appendix-b-financial-statement-basics.qmd
 images/
 quarto-workflow.svg
 editor-layout.svg
 jupyter-notebook-mockup.svg

```

Figure 2.4.: Recommended project folder structure: one file per chapter, plus a shared images folder

A couple of habits worth building now:

- Keep all figures in a single `images/` folder rather than scattering them across the project — it makes the project easier to navigate and easier to back up.
- Give files and images descriptive, lowercase, hyphenated names (e.g., `revenue-by-quarter.png`, not `Figure1_FINAL_v2.png`) — this avoids broken links when you rename things later and matches conventions used in real data projects.

## 2.11. Troubleshooting Common Setup Issues

### ⚠️ “quarto: command not found”

This usually means Quarto wasn’t added to your system’s PATH during installation. Try closing and reopening your terminal (and your code editor) after installing Quarto — this alone fixes the issue most of the time. If it persists, reinstall Quarto and confirm you allowed it to modify your PATH during setup.

### ⚠️ “ModuleNotFoundError: No module named ‘pandas’ ”

Python can’t find the package because it isn’t installed in the environment your editor is using. Run `pip install pandas` in the same terminal your editor uses, and confirm you’re using the Python interpreter you expect (in Positron/VS Code, check the interpreter shown in the bottom status bar).

### ⚠️ PDF rendering fails but HTML works fine

This almost always means the LaTeX installation is missing or incomplete. Run `quarto install tinytex` and try again. If it still fails, re-render with `quarto render --to pdf --log render.log` and check the log file for the specific missing package.

## 2.12. Chapter Summary

- Python, Quarto, and a code editor (Positron or VS Code) are the three tools needed for this course, and each is installed once at the start of the semester.
- A `.py` script, a `.ipynb` notebook, and a `.qmd` Quarto document all run Python code, but serve different purposes: scripts for production code, notebooks for exploration, and Quarto documents for polished, reproducible reports — which is why this book is written entirely in `.qmd` files.
- `quarto render` turns your `.qmd` source into both HTML and PDF from a single source of truth.
- Keeping a consistent project folder structure (chapters at the top level, a shared `images/` folder) will save you time throughout the semester.

## 2.13. Exercises

1. Complete the “Your First Quarto Document” walkthrough in this chapter. Take a screenshot of your rendered HTML output showing the table, and submit it as instructed by your course syllabus.
2. In your own words (2–3 sentences), explain when you would choose to use a Jupyter notebook versus a Quarto document for a piece of analytical work.
3. Intentionally introduce one small error into your `test.qmd` file (e.g., misspell `pandas` as `panadas`), render it, and copy the error message you get. Then fix it and re-render successfully. This exercise is designed to make error messages feel less intimidating early in the course.



## 3. Data Fundamentals for Accountants

### Learning Objectives

By the end of this chapter, you should be able to:

- Explain tidy data principles and identify whether a table is tidy or messy
- Identify the four levels of measurement and match them to common accounting variables
- Recognize the main data types encountered in accounting data (numeric, categorical, date/time, text, boolean)
- Describe the structure and relationship between the chart of accounts, general ledger, trial balance, and subledgers
- Explain the difference between transaction-level and summarized data, and why analytics often needs the former
- Identify common data quality issues in real-world accounting datasets

### 3.1. Why Data Structure Matters Before You Start Coding

It's tempting to jump straight into pandas and start writing code — and we will, starting next chapter. But most real-world accounting analytics problems are **data problems**, not coding problems. A well-written line of Python code applied to poorly structured or misunderstood data will still give you a wrong (or misleading) answer.

This chapter builds the conceptual foundation you need before touching code: what “well-structured” data looks like, what kinds of data you’ll encounter, how core accounting datasets are organized, and what tends to go wrong with real data. Think of this as learning to read a map before you start driving.

### 3.2. Tidy Data Principles

**Tidy data** is a simple, widely-used standard for organizing tabular data so that it's easy to analyze. A dataset is tidy when:

### 3. Data Fundamentals for Accountants

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

This sounds abstract, so let's look at an example. Suppose you have quarterly revenue by product line.

**Messy (wide) version** — each quarter is its own column:

| product_line | Q1     | Q2     | Q3     | Q4     |
|--------------|--------|--------|--------|--------|
| Consulting   | 42,000 | 45,500 | 47,200 | 50,100 |
| Software     | 61,000 | 58,000 | 63,500 | 70,200 |

This looks fine to read as a human, and it's a completely normal way to present numbers in a report. But it's **not tidy**: “quarter” is a variable that's been spread across four columns instead of living in a single column.

**Tidy (long) version** — the same data, restructured:

| product_line | quarter | revenue |
|--------------|---------|---------|
| Consulting   | Q1      | 42,000  |
| Consulting   | Q2      | 45,500  |
| Consulting   | Q3      | 47,200  |
| Consulting   | Q4      | 50,100  |
| Software     | Q1      | 61,000  |
| Software     | Q2      | 58,000  |
| Software     | Q3      | 63,500  |
| Software     | Q4      | 70,200  |

#### Connect to Practice

Tidy data isn't just an academic preference. In Chapter 4, you'll see that pandas operations — filtering, grouping, plotting — are dramatically easier when your data is tidy. A wide table like the “messy” example above requires extra reshaping steps before you can, say, plot revenue trends across all product lines on one chart.

You don't need to memorize a rulebook here — the practical test is: *if I wanted to filter, group, or plot by any single variable, is that variable sitting in one clearly labeled column?* If yes, you're likely looking at tidy data.



### 3.3. Levels of Measurement

Before you calculate anything — an average, a sum, a comparison — it’s worth asking: *what kind of variable is this, and what operations actually make sense on it?* This is the idea behind **levels of measurement**, a classification used throughout statistics.

There are four levels, each allowing more mathematical operations than the last:

| Level           | Description                                                          | Accounting Example                                               | Meaningful Operations                                 |
|-----------------|----------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------|
| <b>Nominal</b>  | Categories with no inherent order                                    | Account type (Asset, Liability, Equity), Department, Vendor name | Count, mode, group by                                 |
| <b>Ordinal</b>  | Ordered categories, but differences between levels aren’t meaningful | Credit risk rating (Low/Medium/High), Audit opinion severity     | Count, mode, median, ranking                          |
| <b>Interval</b> | Meaningful differences, but no true zero                             | Fiscal year, Credit score (in some scales)                       | Addition/subtraction, mean, but not meaningful ratios |
| <b>Ratio</b>    | True zero point; all arithmetic is meaningful                        | Account balance, Transaction amount, Units sold                  | Mean, sum, ratios, all standard arithmetic            |

#### ! Why This Matters in Practice

It’s easy to make a technically-valid-looking calculation that is conceptually meaningless. For example: averaging a customer’s credit risk rating of “Low, Medium, High” by assigning them 1, 2, 3 and computing a mean of 1.8 *looks* like a real number, but “1.8” doesn’t correspond to any real risk category — ordinal data doesn’t support that kind of averaging. Similarly, summing account numbers from your chart of accounts (nominal data, even though it’s stored as a number) produces a number with no accounting meaning at all. We’ll return to this idea in Chapter 5, where choosing the right summary statistic depends directly on a variable’s level of measurement.

## 3.4. Data Types You'll Encounter

Related to levels of measurement, but more about how data is *stored and represented* in a dataset, are data types. In accounting data, you'll regularly see:

- **Numeric — continuous:** dollar amounts (account balances, transaction amounts) — ratio-level, all arithmetic makes sense
- **Numeric — discrete:** counts (units sold, number of invoices) — ratio-level, but only whole numbers make sense
- **Categorical:** account type, department, cost center, vendor — typically nominal, sometimes ordinal (e.g., risk tier)
- **Date/time:** transaction date, fiscal period, posting date — deserves special attention, covered next
- **Text/string:** transaction descriptions, memo fields, vendor names — often messy and inconsistently formatted
- **Boolean/flag:** `is_reversed`, `is_intercompany`, `is_reconciled` — true/false indicators, nominal with only two categories

### 3.4.1. A Special Note on Dates

Dates cause more headaches in accounting data than almost any other data type, for a few reasons:

- They can be stored as **text** ("01/15/2025") instead of an actual date type, which prevents sorting and date arithmetic from working correctly
- **Format ambiguity:** is 03/04/2025 March 4th or April 3rd? This depends on regional conventions (US vs. most of the rest of the world) and is a common source of silent errors
- **Fiscal year vs. calendar year:** a company with a fiscal year ending June 30 will label transactions in "FY2025" that occurred in *calendar* 2024

We'll handle dates properly with pandas in Chapter 4 — for now, just know that a date column that "looks fine" when you eyeball a spreadsheet is one of the most common sources of hidden bugs in accounting analytics.

## 3.5. Core Accounting Data Structures

Most accounting analytics work touches one or more of four related data structures. Figure [3.1](#) shows how they connect.

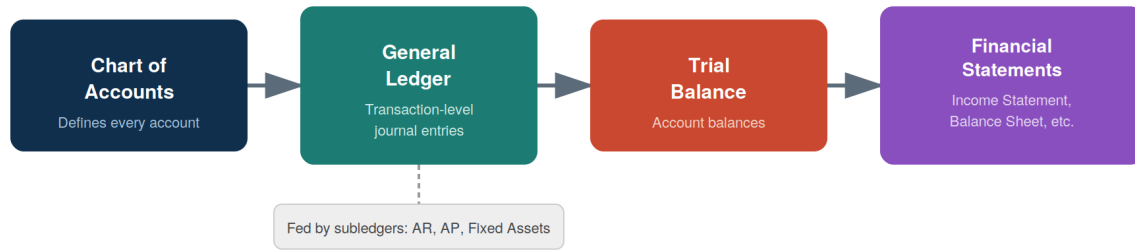


Figure 3.1.: How core accounting data structures relate: the chart of accounts defines every account, transactions flow through the general ledger (fed by subledgers), and roll up into a trial balance and ultimately the financial statements

#### 3.5.1. Chart of Accounts (COA)

The chart of accounts is the master list of every account a company uses — essentially a data dictionary for the rest of the accounting system. A typical COA entry includes an account number, account name, and account type (Asset, Liability, Equity, Revenue, Expense).

#### 3.5.2. General Ledger (GL)

The general ledger contains **transaction-level detail** — every individual journal entry, each with a debit and a credit side, a date, a description, and (typically) the account(s) affected. This is the most granular, information-rich accounting dataset, and the one most audit and fraud analytics work is performed on.

#### 3.5.3. Subsidiary Ledgers

Subsidiary ledgers (Accounts Receivable, Accounts Payable, Fixed Assets, Payroll, etc.) hold detailed records for a specific area that then roll up into summary entries in the general ledger. For example, the AR subledger tracks every individual customer invoice and payment, but the GL might only see a periodic summary entry.

#### 3.5.4. Trial Balance

The trial balance is a **summarized snapshot**: the ending balance of every account in the chart of accounts, as of a point in time. This is the level of detail you worked with in Chapter 2 — one

row per account, not one row per transaction. A trial balance is a checkpoint used to confirm that total debits equal total credits before financial statements are prepared.

## 3.6. Transaction-Level vs. Summarized Data

This distinction matters more than it might first appear:

- A **trial balance** can look completely normal — accounts balanced, nothing unusual — while individual **transactions** underneath it hide errors, unusual patterns, or fraud.
- For example, imagine a “Travel Expense” account with a perfectly reasonable ending balance of \$45,000 for the year. That summary number gives no indication that \$38,000 of it came from a single suspicious transaction posted on a Sunday by an employee who doesn’t normally have travel expenses.

This is exactly why audit analytics (Chapter 8) and fraud detection (Chapter 9) typically require **transaction-level general ledger data**, not just trial balances — the interesting patterns often only show up at the most granular level.

## 3.7. Common Data Quality Issues in Accounting Data

Real accounting data is rarely clean. Some of the most common issues you’ll encounter:

- **Missing values:** blank department codes, missing dates, unpopulated memo fields
- **Duplicate entries:** the same transaction entered twice, sometimes with slightly different formatting
- **Inconsistent formatting:** dates stored as text in different formats, currency values stored with \$ symbols or commas (which prevents numeric calculations until cleaned), inconsistent capitalization in account or vendor names ("Cash" vs. "cash" vs. "CASH " with trailing whitespace)
- **Inconsistent categorical labels:** the same department entered as "HR", "H.R.", and "Human Resources" across different rows
- **Outliers vs. errors:** an unusually large transaction might be a legitimate one-time event (a building purchase) or a data entry error (an extra zero) — these require judgment, not just a statistical rule, to tell apart

### A Realistic Messy Example

Imagine you received this general ledger excerpt exactly as extracted from a client’s system:

| entry_date | account | debit      | department |
|------------|---------|------------|------------|
| 01/15/2025 | Cash    | \$4,500.00 | Sales      |
| 1-15-2025  | cash    | 4500       | sales      |
| 01/15/2025 | Cash    | 4,500      |            |
| 01/32/2025 | Cash    | 45,000     | Sales      |

This tiny sample already contains: inconsistent date formats, inconsistent account name capitalization, inconsistent currency formatting, a missing department value, a likely duplicate entry, an impossible date (01/32/2025), and a value that may be a data-entry error (an extra zero turning \$4,500 into \$45,000). Spotting these issues *before* you calculate anything is a core skill — we’ll clean data like this hands-on in Chapter 4.

### 3.8. A Preview Look at a Real Dataset

To connect these ideas to something concrete, look at the sample transaction-level general ledger below (`general_ledger_sample.csv`), covering two weeks of journal entries for a small company. Each journal entry (`entry_id`) has at least two rows — one debit, one credit — matching how double-entry accounting actually works.

| entry_id | entry_date | account             | account_type | description               | debit | credit | department | posted_by |
|----------|------------|---------------------|--------------|---------------------------|-------|--------|------------|-----------|
| JE1001   | 2025-01-03 | Cash                | Asset        | Customer payment received | 4500  | 0      | Sales      | jsmith    |
| JE1001   | 2025-01-03 | Accounts Receivable | Asset        | Customer payment received | 0     | 4500   | Sales      | jsmith    |
| JE1002   | 2025-01-05 | Inventory           | Asset        | Purchase of merchandise   | 7200  | 0      | Operations | mchen     |
| JE1002   | 2025-01-05 | Accounts Payable    | Liability    | Purchase of merchandise   | 0     | 7200   | Operations | mchen     |

A few things worth noticing just from visual inspection, before we write a single line of pandas

### 3. Data Fundamentals for Accountants

code in Chapter 4:

- This table is **tidy**: each column is a single variable (date, account, debit, credit, etc.), and each row is one line of a journal entry.
- The **level of measurement** varies by column: `debit/credit` are ratio-level (arithmetic is meaningful), `account_type` and `department` are nominal (categories only), `entry_date` is a date type requiring careful handling.
- This is **transaction-level** data — far more granular than the trial balance from Chapter 2 — which is exactly the kind of dataset audit and fraud analytics techniques (Chapters 8–9) are designed to work with.

We'll load and work with this exact file in Chapter 4.

## 3.9. Chapter Summary

- Tidy data — one variable per column, one observation per row, one observational unit per table — makes analysis dramatically easier and is the standard this book (and pandas) assumes.
- Levels of measurement (nominal, ordinal, interval, ratio) determine which calculations are actually meaningful for a given variable — averaging an ordinal rating or summing a nominal code produces numbers without real meaning.
- Accounting data typically falls into a few types: numeric, categorical, date/time, text, and boolean — each with its own quirks, dates especially.
- The chart of accounts, general ledger, subledgers, and trial balance are related but distinct structures, moving from transaction-level detail to summarized snapshots.
- Analytics work — especially audit and fraud analytics — usually requires transaction-level data, since summarized data can hide the patterns that matter most.
- Real accounting data commonly has quality issues: missing values, duplicates, inconsistent formatting, and ambiguous outliers, all of which need to be identified before analysis begins.

## 3.10. Discussion Questions

1. Look back at the “wide” quarterly revenue example early in this chapter. Why might a company's finance team *prefer* the wide format for a printed report, even though it's not tidy for analysis purposes?
2. Give one example (not from this chapter) of an accounting variable at each level of measurement: nominal, ordinal, interval, and ratio.
3. Why might a company's trial balance appear completely normal in a given month, even if fraudulent transactions occurred during that period?

## 3.11. Exercises

1. Take the “messy” wide revenue table from this chapter and manually rewrite it in tidy (long) format on paper or in a spreadsheet.
2. Using the messy general ledger excerpt in the callout box above, list every data quality issue you can find, and for each one, briefly describe how you would investigate or resolve it if this were real client data.
3. Download `general_ledger_sample.csv` (provided alongside this chapter) and open it in a spreadsheet program. Identify: (a) how many unique journal entries it contains, (b) how many unique departments appear, and (c) whether total debits equal total credits.





**Part II.**

## **Part II: Core Data Skills**



## 4. Data Wrangling with Pandas and Polars

### Learning Objectives

By the end of this chapter, you should be able to:

- Understand what data wrangling is and its importance
- Import tabular accounting data (CSV) into both pandas and polars DataFrames
- Select, filter, and sort accounting data using both libraries
- Clean messy accounting data: fix currency formatting, inconsistent text, bad dates, and missing values
- Merge/join accounting tables and explain the difference between inner and left joins
- Reshape data between wide and long formats
- Group and aggregate transaction-level data into account-level summaries
- Explain lazy vs. eager evaluation in polars, and why it matters for large accounting datasets

### 4.1. What is Data Wrangling?

**Data wrangling**—also called *data cleaning*, *data remediation*, or *data munging*—refers to a variety of processes designed to transform raw data into more readily used formats. The exact methods differ from project to project depending on the data you’re leveraging and the goal you’re trying to achieve ([Stobierski, 2021](#)).

Some examples of data wrangling include:

- Merging multiple data sources into a single dataset for analysis
- Identifying gaps in data (for example, empty cells in a spreadsheet) and either filling or deleting them
- Deleting data that’s either unnecessary or irrelevant to the project you’re working on

## 4. Data Wrangling with Pandas and Polars

- Identifying extreme outliers in data and either explaining the discrepancies or removing them so that analysis can take place

Data wrangling can be a manual or automated process. In scenarios where datasets are exceptionally large, automated data cleaning becomes a necessity. In organizations that employ a full data team, a data scientist or other team member is typically responsible for data wrangling. In smaller organizations, non-data professionals are often responsible for cleaning their data before leveraging it (Stobierski, 2021).

### 4.1.1. Data Wrangling Steps and Techniques

Each data project requires a unique approach to ensure its final dataset is reliable and accessible. That being said, several processes typically inform the approach. These are commonly referred to as data wrangling steps or activities. Figure 4.1 shows the steps in Data Wrangling (Stobierski, 2021).



Figure 4.1.: Steps in Data Wrangling

### 4.1.1.1. **Discovery**

Discovery refers to the process of familiarizing yourself with data so you can conceptualize how you might use it. You can liken it to looking in your refrigerator before cooking a meal to see what ingredients you have at your disposal.

During discovery, you may identify trends or patterns in the data, along with obvious issues, such as missing or incomplete values that need to be addressed. This is an important step, as it will inform every activity that comes afterward ([Stobierski, 2021](#)).

### 4.1.1.2. **Structuring**

Raw data is typically unusable in its raw state because it's either incomplete or misformatted for its intended application. Data structuring is the process of taking raw data and transforming it to be more readily leveraged. The form your data takes will depend on the analytical model you use to interpret it ([Stobierski, 2021](#)).

### 4.1.1.3. **Cleaning**

Data cleaning is the process of removing inherent errors in data that might distort your analysis or render it less valuable. Cleaning can come in different forms, including deleting empty cells or rows, removing outliers, and standardizing inputs. The goal of data cleaning is to ensure there are no errors (or as few as possible) that could influence your final analysis. Identifying and removing any bad data greatly impacts the rest of the wrangling processes ([Stobierski, 2021](#)).

### 4.1.1.4. **Enriching**

Once you understand your existing data and have transformed it into a more usable state, you must determine whether you have all of the data necessary for the project at hand. If not, you may choose to enrich or augment your data by incorporating values from other datasets. For this reason, it's important to understand what other data is available for use ([Stobierski, 2021](#)).

If you decide that enrichment is necessary, you need to repeat the steps above for any new data.

### 4.1.1.5. **Validating**

Data validation refers to the process of verifying that your data is both consistent and of a high enough quality. During validation, you may discover issues you need to resolve or conclude that your data is ready to be analyzed. Validation is typically achieved through various automated processes and requires programming ([Stobierski, 2021](#)).

## 4. Data Wrangling with Pandas and Polars

### 4.1.1.6. Publishing

Once your data has been validated, you can publish it. This involves making it available to others within your organization for analysis. The format you use to share the information—such as a written report or electronic file—will depend on your data and the organization’s goals (Stobierski, 2021).

### 4.1.2. Data Wrangling vs. Data Cleaning

Despite the terms being used interchangeably, data wrangling and data cleaning are two different processes. It’s important to make the distinction that data cleaning is a critical step in the data wrangling process to remove inaccurate and inconsistent data. Meanwhile, data-wrangling is the overall process of transforming raw data into a more usable form (Stobierski, 2021). Figure 4.2 compares both.

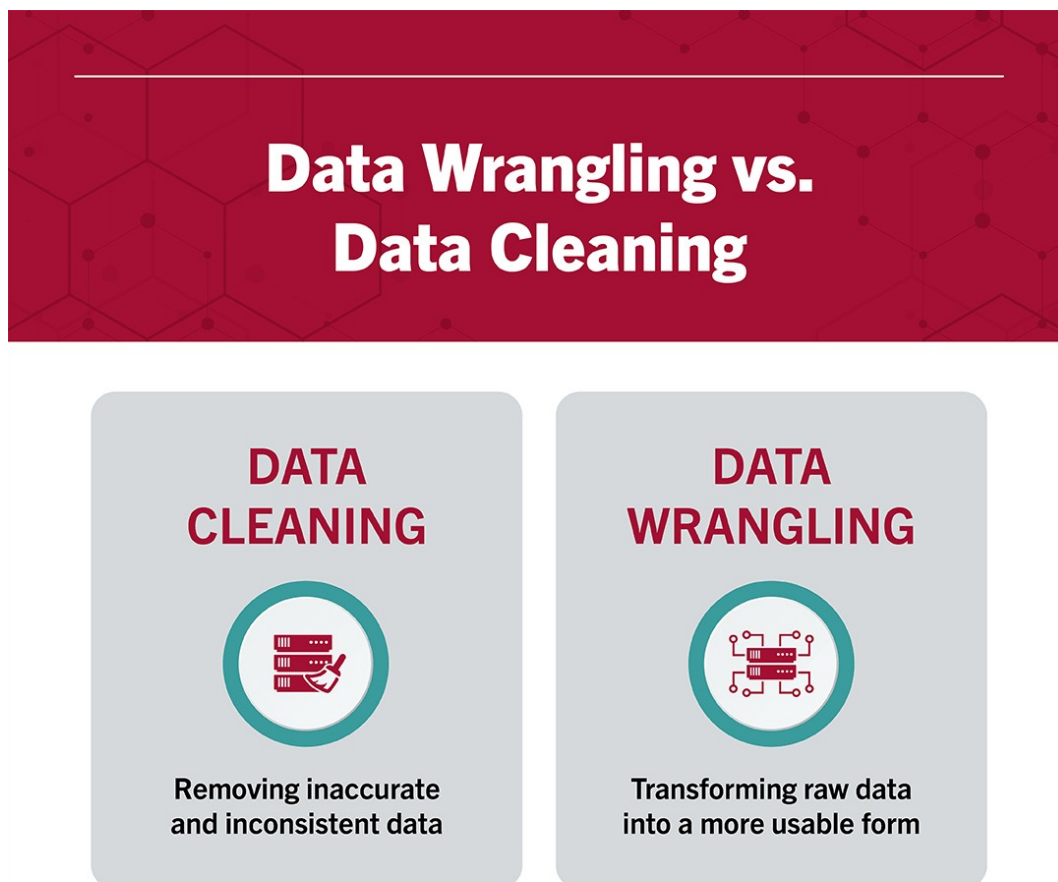


Figure 4.2.: Data Wrangling vs Data Cleaning

### 4.1.3. Importance of Data Wrangling

Any analyses an accountant or CPA performs will ultimately be constrained by the data that informs them. If data is incomplete, unreliable, or faulty, then analyses will be too—diminishing the value of any critical insights gleaned.

Data wrangling seeks to remove that risk by ensuring data is in a reliable state before it's analyzed and leveraged. This makes it a critical part of the analytical process.

It's important to note that data wrangling can be time-consuming and taxing on resources, particularly when done manually. This is why many organizations institute policies and best practices that help employees streamline the data cleanup process—for example, requiring that data include certain information or be in a specific format before it's uploaded to a database.

For this reason, it's vital to understand the steps of the data wrangling process and the negative outcomes associated with incorrect or faulty data ([Stobierski, 2021](#)).

## 4.2. Two Tools, One Set of Concepts

This chapter teaches data wrangling using **two** Python libraries side by side: **pandas**, the long-standing standard for tabular data analysis in Python, and **polars**, a newer library built for speed on large datasets.

The good news: the underlying *concepts* — filtering, joining, grouping, reshaping — are identical no matter which library you use. Only the syntax changes. Learning both at once actually reinforces the concepts, because you'll see the same logic expressed two different ways.

|                                             | pandas                                                      | polars                                                      |
|---------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------|
| <b>Maturity</b>                             | Released 2008; the long-standing standard                   | Released 2020; newer, rapidly growing                       |
| <b>Performance</b>                          | Good for small-to-medium data; single-threaded by default   | Built for speed; multi-threaded, memory-efficient           |
| <b>Syntax style</b>                         | Label-based indexing<br>( <code>df[df["x"] &gt; 0]</code> ) | Expression-based API<br>( <code>pl.col("x") &gt; 0</code> ) |
| <b>Industry adoption (accounting/audit)</b> | Still the dominant tool in most firms and tutorials         | Growing adoption, especially for large transaction datasets |
| <b>Lazy evaluation</b>                      | Not built in (see Dask/Modin for scaling, not covered here) | Built in — covered in this chapter                          |



### Connect to Practice

You are very likely to encounter pandas in an internship or entry-level role today. Polars is worth learning now because its adoption is accelerating quickly, especially anywhere teams work with large general ledger exports (multi-million-row datasets), where its performance advantage becomes very noticeable.

We'll use two datasets you already have from previous chapters — `general_ledger_sample.csv` and `trial_balance.csv` — plus two new ones for this chapter: `chart_of_accounts.csv` (for joins) and `messy_general_ledger.csv` (for cleaning practice).

## 4.3. Reading Data In

Let's start by loading the general ledger sample into both libraries and inspecting its structure.

## 4.4. pandas

```
import pandas as pd
import polars as pl

pandas
gl_pd = pd.read_csv("data/general_ledger_sample.csv")
print(gl_pd.shape)
print(gl_pd.dtypes)
gl_pd.head(3)
```

```
(20, 9)
entry_id str
entry_date str
account str
account_type str
description str
debit int64
credit int64
department str
posted_by str
dtype: object
```



|   | entry_id | entry_date | account             | account_type | description               | debit | credit |
|---|----------|------------|---------------------|--------------|---------------------------|-------|--------|
| 0 | JE1001   | 2025-01-03 | Cash                | Asset        | Customer payment received | 4500  | 0      |
| 1 | JE1001   | 2025-01-03 | Accounts Receivable | Asset        | Customer payment received | 0     | 4500   |
| 2 | JE1002   | 2025-01-05 | Inventory           | Asset        | Purchase of merchandise   | 7200  | 0      |

## 4.5. *polars*

```
polars
gl_pl = pl.read_csv("data/general_ledger_sample.csv")
print(gl_pl.shape)
print(gl_pl.dtypes)
gl_pl.head(3)
```

(20, 9)

[String, String, String, String, String, Int64, Int64, String, String]

| entry_id | entry_date   | account               | account_type | description                 | debit |
|----------|--------------|-----------------------|--------------|-----------------------------|-------|
| str      | str          | str                   | str          | str                         | i64   |
| "JE1001" | "2025-01-03" | "Cash"                | "Asset"      | "Customer payment received" | 4500  |
| "JE1001" | "2025-01-03" | "Accounts Receivable" | "Asset"      | "Customer payment received" | 0     |
| "JE1002" | "2025-01-05" | "Inventory"           | "Asset"      | "Purchase of merchandise"   | 7200  |

Both libraries correctly infer that `debit` and `credit` are numeric and everything else is text. You can get quick summary statistics with `.describe()` in both libraries:

## 4.6. *pandas*

```
gl_pd.describe()
```

|       | debit       | credit      |
|-------|-------------|-------------|
| count | 20.000000   | 20.000000   |
| mean  | 3012.500000 | 3012.500000 |
| std   | 4296.873864 | 4296.873864 |

## 4. Data Wrangling with Pandas and Polars

|     | debit        | credit       |
|-----|--------------|--------------|
| min | 0.000000     | 0.000000     |
| 25% | 0.000000     | 0.000000     |
| 50% | 425.000000   | 425.000000   |
| 75% | 5375.000000  | 5375.000000  |
| max | 15400.000000 | 15400.000000 |

### 4.7. polars

```
gl_pl.describe()
```

| statistic    | entry_id | entry_date   | account             | account_type | description        | debit   |
|--------------|----------|--------------|---------------------|--------------|--------------------|---------|
| str          | str      | str          | str                 | str          | str                | f64     |
| "count"      | "20"     | "20"         | "20"                | "20"         | "20"               | 20.0    |
| "null_count" | "0"      | "0"          | "0"                 | "0"          | "0"                | 0.0     |
| "mean"       | null     | null         | null                | null         | null               | 3012.5  |
| "std"        | null     | null         | null                | null         | null               | 4296.8  |
| "min"        | "JE1001" | "2025-01-03" | "Accounts Payable"  | "Asset"      | "Biweekly payroll" | 0.0     |
| "25%"        | null     | null         | null                | null         | null               | 0.0     |
| "50%"        | null     | null         | null                | null         | null               | 850.0   |
| "75%"        | null     | null         | null                | null         | null               | 5100.0  |
| "max"        | "JE1010" | "2025-01-31" | "Utilities Expense" | "Revenue"    | "Vendor payment"   | 15400.0 |

#### 4.7.1. Reading with an Explicit Schema

By default, both libraries *guess* each column's data type. This usually works, but for accounting data it's good practice to be explicit — especially for ID columns that look numeric but should stay text (an `entry_id` of 1001 should never be treated as a number you could accidentally average).

### 4.8. pandas

```
pandas: force entry_id to stay text
gl_pd2 = pd.read_csv("data/general_ledger_sample.csv", dtype={"entry_id": str})
print(gl_pd2["entry_id"].dtype)
```

str

## 4.9. polars

```
polars: same idea, using schema_overrides
gl_pl2 = pl.read_csv(
 "data/general_ledger_sample.csv",
 schema_overrides={"entry_id": pl.Utf8}
)
print(gl_pl2["entry_id"].dtype)
```

String

### Connect to Practice

Excel files are just as common as CSVs in accounting practice. Both libraries can read them directly: `pd.read_excel("file.xlsx")` and `pl.read_excel("file.xlsx")` (polars requires the `fastexcel` package, installable with `pip install fastexcel`).

## 4.10. Selecting, Filtering, and Sorting

### 4.10.1. Filtering on a Single Condition

Let's find every transaction with a debit over \$5,000 — a common first step in transaction testing.

## 4.11. pandas

```
pandas
large_pd = gl_pd[gl_pd["debit"] > 5000]
large_pd[["entry_id", "account", "debit"]]
```

|    | entry_id | account             | debit |
|----|----------|---------------------|-------|
| 2  | JE1002   | Inventory           | 7200  |
| 4  | JE1003   | Salaries Expense    | 15400 |
| 6  | JE1004   | Accounts Receivable | 9800  |
| 12 | JE1007   | Cash                | 6200  |
| 14 | JE1008   | Cost of Goods Sold  | 5100  |
| 16 | JE1009   | Accounts Payable    | 7200  |

### 4.12. polars

```
polars
large_pl = gl_pl.filter(pl.col("debit") > 5000)
large_pl.select(["entry_id", "account", "debit"])
```

| entry_id | account               | debit |
|----------|-----------------------|-------|
| str      | str                   | i64   |
| "JE1002" | "Inventory"           | 7200  |
| "JE1003" | "Salaries Expense"    | 15400 |
| "JE1004" | "Accounts Receivable" | 9800  |
| "JE1007" | "Cash"                | 6200  |
| "JE1008" | "Cost of Goods Sold"  | 5100  |
| "JE1009" | "Accounts Payable"    | 7200  |

#### 4.12.1. Combining Multiple Conditions

Now let's narrow this to transactions in the Operations department with a nonzero debit — combining two conditions with & (AND).

### 4.13. pandas

```
pandas
combo_pd = gl_pd[(gl_pd["department"] == "Operations") & (gl_pd["debit"] > 0)]
combo_pd[["entry_id", "account", "debit", "department"]]
```

|    | entry_id | account              | debit | department |
|----|----------|----------------------|-------|------------|
| 2  | JE1002   | Inventory            | 7200  | Operations |
| 8  | JE1005   | Rent Expense         | 3000  | Operations |
| 10 | JE1006   | Utilities Expense    | 850   | Operations |
| 14 | JE1008   | Cost of Goods Sold   | 5100  | Operations |
| 16 | JE1009   | Accounts Payable     | 7200  | Operations |
| 18 | JE1010   | Depreciation Expense | 1000  | Operations |

## 4.14. polars

```
polars
combo_pl = gl_pl.filter(
 (pl.col("department") == "Operations") & (pl.col("debit") > 0)
)
combo_pl.select(["entry_id", "account", "debit", "department"])
```

| entry_id | account                | debit | department   |
|----------|------------------------|-------|--------------|
| str      | str                    | i64   | str          |
| "JE1002" | "Inventory"            | 7200  | "Operations" |
| "JE1005" | "Rent Expense"         | 3000  | "Operations" |
| "JE1006" | "Utilities Expense"    | 850   | "Operations" |
| "JE1008" | "Cost of Goods Sold"   | 5100  | "Operations" |
| "JE1009" | "Accounts Payable"     | 7200  | "Operations" |
| "JE1010" | "Depreciation Expense" | 1000  | "Operations" |

### ! A Common Mistake

In pandas, you must use `&` and `|` (not Python's `and/or`) when combining conditions, and each condition needs its own parentheses: `(gl_pd["department"] == "Operations") & (gl_pd["debit"] > 0)`. Forgetting the parentheses is one of the most common pandas errors for beginners.

### 4.14.1. Sorting

Let's sort by debit amount (largest first), breaking ties by date (earliest first):

## 4.15. pandas

```
pandas
gl_pd.sort_values(["debit", "entry_date"], ascending=[False, True]).head()
```

|    | entry_id | entry_date | account             | account_type | description               | debit | c |
|----|----------|------------|---------------------|--------------|---------------------------|-------|---|
| 4  | JE1003   | 2025-01-08 | Salaries Expense    | Expense      | Biweekly payroll          | 15400 | 0 |
| 6  | JE1004   | 2025-01-12 | Accounts Receivable | Asset        | Sale on account           | 9800  | 0 |
| 2  | JE1002   | 2025-01-05 | Inventory           | Asset        | Purchase of merchandise   | 7200  | 0 |
| 16 | JE1009   | 2025-01-29 | Accounts Payable    | Liability    | Vendor payment            | 7200  | 0 |
| 12 | JE1007   | 2025-01-22 | Cash                | Asset        | Customer payment received | 6200  | 0 |

## 4.16. polars

```
polars
gl_pl.sort(["debit", "entry_date"], descending=[True, False]).head()
```

| entry_id | entry_date   | account               | account_type | description                 | debit |
|----------|--------------|-----------------------|--------------|-----------------------------|-------|
| str      | str          | str                   | str          | str                         | i64   |
| "JE1003" | "2025-01-08" | "Salaries Expense"    | "Expense"    | "Biweekly payroll"          | 15400 |
| "JE1004" | "2025-01-12" | "Accounts Receivable" | "Asset"      | "Sale on account"           | 9800  |
| "JE1002" | "2025-01-05" | "Inventory"           | "Asset"      | "Purchase of merchandise"   | 7200  |
| "JE1009" | "2025-01-29" | "Accounts Payable"    | "Liability"  | "Vendor payment"            | 7200  |
| "JE1007" | "2025-01-22" | "Cash"                | "Asset"      | "Customer payment received" | 6200  |

## 4.17. Cleaning Data

Real accounting data is rarely as clean as `general_ledger_sample.csv`. Let's work through `messy_general_ledger.csv`, which has the same kinds of issues introduced in Chapter 3: currency symbols, inconsistent formatting, an impossible date, a missing value, and a likely duplicate entry.

### 4.17.1. Step 1: Fix Currency Formatting

## 4.18. pandas

```
messy_pd = pd.read_csv("data/messy_general_ledger.csv")
messy_pd
```

|    | entry_id | entry_date | account             | debit      | credit     | department |
|----|----------|------------|---------------------|------------|------------|------------|
| 0  | JE2001   | 01/15/2025 | Cash                | \$4,500.00 | 0          | Sales      |
| 1  | JE2001   | 1-15-2025  | cash                | 4500       | 0          | sales      |
| 2  | JE2002   | 01/16/2025 | Accounts Receivable | 0          | \$4,500.00 | Sales      |
| 3  | JE2003   | 01/18/2025 | Inventory           | 7200       | 0          | Operations |
| 4  | JE2003   | 01/18/2025 | Accounts Payable    | 0          | 7200       | Operations |
| 5  | JE2004   | 02/30/2025 | Rent Expense        | 3000       | 0          | Operations |
| 6  | JE2004   | 02/30/2025 | Cash                | 0          | 3000       | Operations |
| 7  | JE2005   | 01/22/2025 | SALARIES EXPENSE    | 15400      | 0          | NaN        |
| 8  | JE2005   | 01/22/2025 | Cash                | 0          | 15400      | HR         |
| 9  | JE2006   | 01/25/2025 | Utilities Expense   | 850        | 0          | Operations |
| 10 | JE2006   | 01/25/2025 | Accounts Payable    | 0          | 8500       | Operations |

```
pandas
df_pd = messy_pd.copy()
for col in ["debit", "credit"]:
 df_pd[col] = (
 df_pd[col].astype(str)
 .str.replace("$", "", regex=False)
 .str.replace(",", "", regex=False)
)
 df_pd[col] = pd.to_numeric(df_pd[col])

df_pd[["entry_id", "debit", "credit"]]
```

|   | entry_id | debit  | credit |
|---|----------|--------|--------|
| 0 | JE2001   | 4500.0 | 0.0    |
| 1 | JE2001   | 4500.0 | 0.0    |
| 2 | JE2002   | 0.0    | 4500.0 |
| 3 | JE2003   | 7200.0 | 0.0    |

|    | entry_id | debit   | credit  |
|----|----------|---------|---------|
| 4  | JE2003   | 0.0     | 7200.0  |
| 5  | JE2004   | 3000.0  | 0.0     |
| 6  | JE2004   | 0.0     | 3000.0  |
| 7  | JE2005   | 15400.0 | 0.0     |
| 8  | JE2005   | 0.0     | 15400.0 |
| 9  | JE2006   | 850.0   | 0.0     |
| 10 | JE2006   | 0.0     | 8500.0  |

## 4.19. polars

```
polars
messy_pl = pl.read_csv("data/messy_general_ledger.csv")

df_pl = messy_pl.with_columns([
 pl.col("debit").str.replace_all(r"\$", "").str.replace_all(",", "").cast(pl.F64),
 pl.col("credit").str.replace_all(r"\$", "").str.replace_all(",", "").cast(pl.F64)
])
df_pl.select(["entry_id", "debit", "credit"])
```

| entry_id | debit   | credit  |
|----------|---------|---------|
| str      | f64     | f64     |
| "JE2001" | 4500.0  | 0.0     |
| "JE2001" | 4500.0  | 0.0     |
| "JE2002" | 0.0     | 4500.0  |
| "JE2003" | 7200.0  | 0.0     |
| "JE2003" | 0.0     | 7200.0  |
| ...      | ...     | ...     |
| "JE2004" | 0.0     | 3000.0  |
| "JE2005" | 15400.0 | 0.0     |
| "JE2005" | 0.0     | 15400.0 |
| "JE2006" | 850.0   | 0.0     |
| "JE2006" | 0.0     | 8500.0  |

The `debit` and `credit` columns contain `$` and `,` characters, which force them to be read as text instead of numbers.



### 4.19.1. Step 2: Standardize Text Fields

The account and department columns have inconsistent capitalization and stray whitespace.

## 4.20. pandas

```
pandas
df_pd["account"] = df_pd["account"].str.strip().str.title()
df_pd["department"] = df_pd["department"].str.strip().str.title()
df_pd[["account", "department"]]
```

|    | account             | department |
|----|---------------------|------------|
| 0  | Cash                | Sales      |
| 1  | Cash                | Sales      |
| 2  | Accounts Receivable | Sales      |
| 3  | Inventory           | Operations |
| 4  | Accounts Payable    | Operations |
| 5  | Rent Expense        | Operations |
| 6  | Cash                | Operations |
| 7  | Salaries Expense    | NaN        |
| 8  | Cash                | Hr         |
| 9  | Utilities Expense   | Operations |
| 10 | Accounts Payable    | Operations |

## 4.21. polars

```
polars
df_pl = df_pl.with_columns([
 pl.col("account").str.strip_chars().str.to_titlecase(),
 pl.col("department").str.strip_chars().str.to_titlecase(),
])
df_pl.select(["account", "department"])
```

## 4. Data Wrangling with Pandas and Polars

| account<br>str        | department<br>str |
|-----------------------|-------------------|
| "Cash"                | "Sales"           |
| "Cash"                | "Sales"           |
| "Accounts Receivable" | "Sales"           |
| "Inventory"           | "Operations"      |
| "Accounts Payable"    | "Operations"      |
| ...                   | ...               |
| "Cash"                | "Operations"      |
| "Salaries Expense"    | null              |
| "Cash"                | "Hr"              |
| "Utilities Expense"   | "Operations"      |
| "Accounts Payable"    | "Operations"      |

### ⚠ Automated Cleaning Still Needs a Human Check

Look closely at the department column: "HR" became "Hr" after title-casing, since `.title()` (and its polars equivalent) simply capitalizes the first letter of each word — it doesn't know HR is an acronym that should stay fully capitalized. This is a good example of why you should always spot-check the results of automated cleaning steps rather than assuming they worked perfectly. A more robust fix here would be an explicit mapping (e.g., `{"hr": "HR", "Hr": "HR"}`) for known acronyms in your data.

### 4.21.1. Step 3: Fix Inconsistent Dates

The `entry_date` column mixes 01/15/2025 and 1-15-2025 formats, and contains one impossible date (02/30/2025 — February never has 30 days).

## 4.22. pandas

```
pandas: format="mixed" handles multiple formats; errors="coerce" turns bad date
df_pd["entry_date"] = pd.to_datetime(df_pd["entry_date"], format="mixed", errors=
df_pd[["entry_id", "entry_date"]]
```

|   | entry_id | entry_date |
|---|----------|------------|
| 0 | JE2001   | 2025-01-15 |

|    | entry_id | entry_date |
|----|----------|------------|
| 1  | JE2001   | 2025-01-15 |
| 2  | JE2002   | 2025-01-16 |
| 3  | JE2003   | 2025-01-18 |
| 4  | JE2003   | 2025-01-18 |
| 5  | JE2004   | NaT        |
| 6  | JE2004   | NaT        |
| 7  | JE2005   | 2025-01-22 |
| 8  | JE2005   | 2025-01-22 |
| 9  | JE2006   | 2025-01-25 |
| 10 | JE2006   | 2025-01-25 |

```
pandas: find the rows that failed to parse
df_pd[df_pd["entry_date"].isna()]
```

|   | entry_id | entry_date | account      | debit  | credit | department |
|---|----------|------------|--------------|--------|--------|------------|
| 5 | JE2004   | NaT        | Rent Expense | 3000.0 | 0.0    | Operations |
| 6 | JE2004   | NaT        | Cash         | 0.0    | 3000.0 | Operations |

## 4.23. **polars**

Polars doesn't have an automatic “mixed format” parser, so we normalize the separators first, then parse with `strict=False` so invalid dates become null instead of raising an error.

```
polars
df_pl = df_pl.with_columns(
 pl.col("entry_date")
 .str.replace_all("-", "/")
 .str.to_date(format="%m/%d/%Y", strict=False)
 .alias("entry_date")
)
df_pl.select(["entry_id", "entry_date"])
```

| entry_id | entry_date |
|----------|------------|
| str      | date       |
| "JE2001" | 2025-01-15 |

## 4. Data Wrangling with Pandas and Polars

| entry_id<br>str | entry_date<br>date |
|-----------------|--------------------|
| "JE2001"        | 2025-01-15         |
| "JE2002"        | 2025-01-16         |
| "JE2003"        | 2025-01-18         |
| "JE2003"        | 2025-01-18         |
| ...             | ...                |
| "JE2004"        | null               |
| "JE2005"        | 2025-01-22         |
| "JE2005"        | 2025-01-22         |
| "JE2006"        | 2025-01-25         |
| "JE2006"        | 2025-01-25         |

### ! What Should You Do With an Invalid Date?

Notice that 02/30/2025 became a missing value (NaT in pandas, null in polars) rather than an error that stops your code. This is intentional — but it means *you* now have to decide what to do about it: go back to the source system to find the correct date, exclude the transaction, or flag it for follow-up. Silently ignoring nulls at this point is how real errors slip through into financial reports.

### 4.23.1. Step 4: Handle Missing Values

The department column has one missing value.

## 4.24. pandas

```
pandas
df_pd["department"] = df_pd["department"].fillna("Unassigned")
df_pd[["entry_id", "department"]]
```

|   | entry_id | department |
|---|----------|------------|
| 0 | JE2001   | Sales      |
| 1 | JE2001   | Sales      |
| 2 | JE2002   | Sales      |
| 3 | JE2003   | Operations |

|    | entry_id | department |
|----|----------|------------|
| 4  | JE2003   | Operations |
| 5  | JE2004   | Operations |
| 6  | JE2004   | Operations |
| 7  | JE2005   | Unassigned |
| 8  | JE2005   | Hr         |
| 9  | JE2006   | Operations |
| 10 | JE2006   | Operations |

## 4.25. polars

```
polars
df_pl = df_pl.with_columns(
 pl.col("department").fill_null("Unassigned")
)
df_pl.select(["entry_id", "department"])
```

| entry_id | department   |
|----------|--------------|
| str      | str          |
| "JE2001" | "Sales"      |
| "JE2001" | "Sales"      |
| "JE2002" | "Sales"      |
| "JE2003" | "Operations" |
| "JE2003" | "Operations" |
| ...      | ...          |
| "JE2004" | "Operations" |
| "JE2005" | "Unassigned" |
| "JE2005" | "Hr"         |
| "JE2006" | "Operations" |
| "JE2006" | "Operations" |

### 4.25.1. Step 5: Check for Duplicates

After cleaning, compare rows on the fields that should uniquely identify a real transaction line.

### 4.26. pandas

```
pandas
dupes_pd = df_pd[df_pd.duplicated(subset=["account", "debit", "credit"], keep=False)]
dupes_pd
```

|   | entry_id | entry_date | account | debit  | credit | department |
|---|----------|------------|---------|--------|--------|------------|
| 0 | JE2001   | 2025-01-15 | Cash    | 4500.0 | 0.0    | Sales      |
| 1 | JE2001   | 2025-01-15 | Cash    | 4500.0 | 0.0    | Sales      |

### 4.27. polars

```
dupes_pl = df_pl.filter(df_pl.is_duplicated())
dupes_pl
```

| entry_id | entry_date | account | debit  | credit | department |
|----------|------------|---------|--------|--------|------------|
| str      | date       | str     | f64    | f64    | str        |
| "JE2001" | 2025-01-15 | "Cash"  | 4500.0 | 0.0    | "Sales"    |
| "JE2001" | 2025-01-15 | "Cash"  | 4500.0 | 0.0    | "Sales"    |

The two JE2001 rows for Cash are now revealed as duplicates once formatting differences are removed — before cleaning, "cash " (lowercase, trailing space) looked like a different value from "Cash", hiding the duplicate.

### 4.28. Merging and Joining Tables

Accounting data rarely lives in a single table. Let's enrich the general ledger with account type information from `chart_of_accounts.csv`.

### 4.29. pandas

```
coa_pd = pd.read_csv("data/chart_of_accounts.csv")
coa_pd.head()
```

|   | account             | account_number | account_type |
|---|---------------------|----------------|--------------|
| 0 | Cash                | 1000           | Asset        |
| 1 | Accounts Receivable | 1100           | Asset        |
| 2 | Inventory           | 1200           | Asset        |
| 3 | Prepaid Insurance   | 1300           | Asset        |
| 4 | Equipment           | 1500           | Asset        |

## 4.30. polars

```
polars
coa_pl = pl.read_csv("data/chart_of_accounts.csv")
```

### 4.30.1. Inner Join

An **inner join** keeps only rows where the join key (account) matches in *both* tables.

## 4.31. pandas

```
pandas
inner_pd = gl_pd.merge(coa_pd, on="account", how="inner")
print(inner_pd.shape)
sorted(inner_pd["account"].unique())
```

```
(19, 11)
```

```
['Accounts Payable',
 'Accounts Receivable',
 'Accumulated Depreciation - Equipment',
 'Cash',
 'Cost of Goods Sold',
```

## 4. Data Wrangling with Pandas and Polars

```
'Depreciation Expense',
'Inventory',
'Rent Expense',
'Salaries Expense',
'Sales Revenue']
```

### 4.32. polars

```
polars
inner_pl = gl_pl.join(coa_pl, on="account", how="inner")
print(inner_pl.shape)
```

(19, 11)

Notice the inner join result has fewer rows than the original general ledger (19 vs. 20). One account is missing.

#### 4.32.1. Left Join

A **left join** keeps *every* row from the left table (the general ledger), filling in null/NaN where there's no match in the chart of accounts.

### 4.33. pandas

```
pandas
left_pd = gl_pd.merge(coa_pd, on="account", how="left")
missing_pd = left_pd[left_pd["account_number"].isna()]
missing_pd[["entry_id", "account"]].drop_duplicates()
```

|    | entry_id | account           |
|----|----------|-------------------|
| 10 | JE1006   | Utilities Expense |



## 4.34. polars

```
polars
left_pl = gl_pl.join(coa_pl, on="account", how="left")
missing_pl = left_pl.filter(pl.col("account_number").is_null())
missing_pl.select(["entry_id", "account"]).unique()
```

| entry_id | account             |
|----------|---------------------|
| str      | str                 |
| "JE1006" | "Utilities Expense" |

Both approaches reveal the same finding: **"Utilities Expense"** appears in the general ledger but not in the chart of accounts — likely a new account that hasn't been added to the COA yet, or a naming mismatch (e.g., "Utilities" vs. "Utilities Expense") that needs investigation.

### ! Why This Matters

An inner join would have silently *dropped* the Utilities Expense transactions from any analysis without telling you — the row count would just be smaller, with no error or warning. A left join preserves every general ledger transaction and makes the mismatch visible. When enriching transaction data with reference tables, prefer a left join and explicitly check for unmatched rows, rather than defaulting to an inner join.

## 4.35. Reshaping Data

Recall the tidy data discussion from Chapter 3. Let's build a **wide** summary table — one common way accountants like to view data — and then reshape it back to **long** (tidy) format.

### 4.35.1. Wide: Total Debits by Account and Department

## 4.36. pandas

```
pandas
wide_pd = gl_pd.pivot_table(
 index="account", columns="department", values="debit",
```

#### 4. Data Wrangling with Pandas and Polars

```
aggfunc="sum", fill_value=0
)
wide_pd
```

| department                           | HR    | Operations | Sales |
|--------------------------------------|-------|------------|-------|
| account                              |       |            |       |
| Accounts Payable                     | 0     | 7200       | 0     |
| Accounts Receivable                  | 0     | 0          | 9800  |
| Accumulated Depreciation - Equipment | 0     | 0          | 0     |
| Cash                                 | 0     | 0          | 10700 |
| Cost of Goods Sold                   | 0     | 5100       | 0     |
| Depreciation Expense                 | 0     | 1000       | 0     |
| Inventory                            | 0     | 7200       | 0     |
| Rent Expense                         | 0     | 3000       | 0     |
| Salaries Expense                     | 15400 | 0          | 0     |
| Sales Revenue                        | 0     | 0          | 0     |
| Utilities Expense                    | 0     | 850        | 0     |

### 4.37. polars

```
polars
wide_pl = gl_pl.pivot(
 index="account", on="department", values="debit",
 aggregate_function="sum"
).fill_null(0)
wide_pl
```

| account               | Sales | Operations | HR    |
|-----------------------|-------|------------|-------|
| str                   | i64   | i64        | i64   |
| "Cash"                | 10700 | 0          | 0     |
| "Accounts Receivable" | 9800  | 0          | 0     |
| "Inventory"           | 0     | 7200       | 0     |
| "Accounts Payable"    | 0     | 7200       | 0     |
| "Salaries Expense"    | 0     | 0          | 15400 |
| ...                   | ...   | ...        | ...   |
| "Rent Expense"        | 0     | 3000       | 0     |

| account<br>str                      | Sales<br>i64 | Operations<br>i64 | HR<br>i64 |
|-------------------------------------|--------------|-------------------|-----------|
| "Utilities Expense"                 | 0            | 850               | 0         |
| "Cost of Goods Sold"                | 0            | 5100              | 0         |
| "Depreciation Expense"              | 0            | 1000              | 0         |
| "Accumulated Depreciation - Equ..." | 0            | 0                 | 0         |

### 4.37.1. Back to Long (Tidy) Format

## 4.38. pandas

```
pandas
long_pd = wide_pd.reset_index().melt(
 id_vars="account", var_name="department", value_name="debit"
)
long_pd.head()
```

|   | account                              | department | debit |
|---|--------------------------------------|------------|-------|
| 0 | Accounts Payable                     | HR         | 0     |
| 1 | Accounts Receivable                  | HR         | 0     |
| 2 | Accumulated Depreciation - Equipment | HR         | 0     |
| 3 | Cash                                 | HR         | 0     |
| 4 | Cost of Goods Sold                   | HR         | 0     |

## 4.39. polars

```
polars
long_pl = wide_pl.unpivot(
 index="account", variable_name="department", value_name="debit"
)
long_pl.head()
```

## 4. Data Wrangling with Pandas and Polars

| account<br>str        | department<br>str | debit<br>i64 |
|-----------------------|-------------------|--------------|
| "Cash"                | "Sales"           | 10700        |
| "Accounts Receivable" | "Sales"           | 9800         |
| "Inventory"           | "Sales"           | 0            |
| "Accounts Payable"    | "Sales"           | 0            |
| "Salaries Expense"    | "Sales"           | 0            |

Both directions matter in practice: the wide format is easy for a human to scan in a report, while the long/tidy format is what you want as an input to further analysis, grouping, or plotting (Chapter 6).

### 4.40. Grouping and Aggregating

One of the most common accounting analytics tasks is summarizing transaction-level data into account-level totals — in other words, **rebuilding a trial balance from the general ledger**.

### 4.41. pandas

```
pandas
tb_pd = gl_pd.groupby("account", as_index=False)[["debit", "credit"]].sum()
tb_pd
```

|    | account                              | debit | credit |
|----|--------------------------------------|-------|--------|
| 0  | Accounts Payable                     | 7200  | 8050   |
| 1  | Accounts Receivable                  | 9800  | 10700  |
| 2  | Accumulated Depreciation - Equipment | 0     | 1000   |
| 3  | Cash                                 | 10700 | 25600  |
| 4  | Cost of Goods Sold                   | 5100  | 0      |
| 5  | Depreciation Expense                 | 1000  | 0      |
| 6  | Inventory                            | 7200  | 5100   |
| 7  | Rent Expense                         | 3000  | 0      |
| 8  | Salaries Expense                     | 15400 | 0      |
| 9  | Sales Revenue                        | 0     | 9800   |
| 10 | Utilities Expense                    | 850   | 0      |

```
print("Total debits:", tb_pd["debit"].sum())
print("Total credits:", tb_pd["credit"].sum())
```

```
Total debits: 60250
Total credits: 60250
```

## 4.42. polars

```
polars
tb_pl = (
 gl_pl.group_by("account")
 .agg([pl.col("debit").sum(), pl.col("credit").sum()])
 .sort("account")
)
tb_pl
```

| account                            | debit | credit |
|------------------------------------|-------|--------|
| str                                | i64   | i64    |
| "Accounts Payable"                 | 7200  | 8050   |
| "Accounts Receivable"              | 9800  | 10700  |
| "Accumulated Depreciation - Equ... | 0     | 1000   |
| "Cash"                             | 10700 | 25600  |
| "Cost of Goods Sold"               | 5100  | 0      |
| ...                                | ...   | ...    |
| "Inventory"                        | 7200  | 5100   |
| "Rent Expense"                     | 3000  | 0      |
| "Salaries Expense"                 | 15400 | 0      |
| "Sales Revenue"                    | 0     | 9800   |
| "Utilities Expense"                | 850   | 0      |

Notice that total debits equal total credits (\$60,250 each) — exactly what we’d expect, since this confirms the sample general ledger is internally balanced, the same check you’d run on any real trial balance.

### 4.42.1. Multi-Level Grouping

You can group by more than one column at once — for example, account *within* department:

## 4.43. pandas

```
pandas
multi_pd = gl_pd.groupby(["department", "account"], as_index=False)[["debit", "credit"]]
multi_pd
```

|    | department | account                              | debit | credit |
|----|------------|--------------------------------------|-------|--------|
| 0  | HR         | Cash                                 | 0     | 15400  |
| 1  | HR         | Salaries Expense                     | 15400 | 0      |
| 2  | Operations | Accounts Payable                     | 7200  | 8050   |
| 3  | Operations | Accumulated Depreciation - Equipment | 0     | 1000   |
| 4  | Operations | Cash                                 | 0     | 10200  |
| 5  | Operations | Cost of Goods Sold                   | 5100  | 0      |
| 6  | Operations | Depreciation Expense                 | 1000  | 0      |
| 7  | Operations | Inventory                            | 7200  | 5100   |
| 8  | Operations | Rent Expense                         | 3000  | 0      |
| 9  | Operations | Utilities Expense                    | 850   | 0      |
| 10 | Sales      | Accounts Receivable                  | 9800  | 10700  |
| 11 | Sales      | Cash                                 | 10700 | 0      |
| 12 | Sales      | Sales Revenue                        | 0     | 9800   |

## 4.44. polars

```
polars
multi_pl = (
 gl_pl.group_by(["department", "account"])
 .agg([pl.col("debit").sum(), pl.col("credit").sum()])
 .sort(["department", "account"])
)
multi_pl
```

| department   | account            | debit | credit |
|--------------|--------------------|-------|--------|
| str          | str                | i64   | i64    |
| "HR"         | "Cash"             | 0     | 15400  |
| "HR"         | "Salaries Expense" | 15400 | 0      |
| "Operations" | "Accounts Payable" | 7200  | 8050   |

| department   | account                             | debit | credit |
|--------------|-------------------------------------|-------|--------|
| str          | str                                 | i64   | i64    |
| "Operations" | "Accumulated Depreciation - Equ..." | 0     | 1000   |
| "Operations" | "Cash"                              | 0     | 10200  |
| ...          | ...                                 | ...   | ...    |
| "Operations" | "Rent Expense"                      | 3000  | 0      |
| "Operations" | "Utilities Expense"                 | 850   | 0      |
| "Sales"      | "Accounts Receivable"               | 9800  | 10700  |
| "Sales"      | "Cash"                              | 10700 | 0      |
| "Sales"      | "Sales Revenue"                     | 0     | 9800   |

## 4.45. Polars Lazy Frames: Wrangling Large Accounting Datasets

Everything so far has used **eager evaluation** — each line of code runs immediately, and the result is fully computed and stored in memory before you move to the next step. This is how pandas always works, and how polars works by default with `pl.read_csv()`.

Polars also offers **lazy evaluation**: instead of running each step immediately, you build up a *query plan* describing everything you want to do, and only execute it once, at the end, with `.collect()`. This matters enormously for large datasets, because polars can look at your *entire* pipeline before running anything and optimize it — for example, reading only the columns you actually use, or applying filters before an expensive join instead of after.

### 4.45.1. Eager vs. Lazy: The Same Pipeline, Two Ways

Let's take a realistic pipeline — filter to nonzero debits, join to the chart of accounts, then total debits by account type — and run it both ways.

```
EAGER: every step executes immediately and materializes a full result in memory
result_eager = (
 gl_pl
 .filter(pl.col("debit") > 0)
 .join(coa_pl, on="account", how="inner")
 .group_by("account_type")
 .agg(pl.col("debit").sum().alias("total_debit"))
 .sort("account_type")
)
```

#### 4. Data Wrangling with Pandas and Polars

```
)
result_eager
```

| account_type<br>str | total_debit<br>i64 |
|---------------------|--------------------|
| "Asset"             | 27700              |
| "Expense"           | 24500              |
| "Liability"         | 7200               |

```
LAZY: scan_csv builds a query plan; nothing runs until .collect()
lazy_query = (
 pl.scan_csv("data/general_ledger_sample.csv")
 .filter(pl.col("debit") > 0)
 .join(pl.scan_csv("data/chart_of_accounts.csv"), on="account", how="inner")
 .group_by("account_type")
 .agg(pl.col("debit").sum().alias("total_debit"))
 .sort("account_type")
)

result_lazy = lazy_query.collect()
result_lazy
```

| account_type<br>str | total_debit<br>i64 |
|---------------------|--------------------|
| "Asset"             | 27700              |
| "Expense"           | 24500              |
| "Liability"         | 7200               |

Both approaches give an identical result — but they got there differently. You can inspect *how* polars plans to execute the lazy version before running it, using `.explain()`:

```
print(lazy_query.explain())
```

```
SORT BY [col("account_type")]
AGGREGATE[maintain_order: false]
 [col("debit").sum().alias("total_debit")] BY [col("account_type")]
FROM
```



```

simple 2/2 ["account_type", "debit"]
 INNER JOIN:
 LEFT PLAN ON: [col("account")]
 simple 3/3 ["account_type", "debit", ... 1 other column]
 Csv SCAN [data/general_ledger_sample.csv]
 PROJECT 3/9 COLUMNS
 SELECTION: [(col("debit")) > (0)]
 ESTIMATED ROWS: 23
 RIGHT PLAN ON: [col("account")]
 Csv SCAN [data/chart_of_accounts.csv]
 PROJECT 1/3 COLUMNS
 ESTIMATED ROWS: 37
 END INNER JOIN

```

Look closely at the plan: notice it shows `PROJECT 3/9 COLUMNS` for the general ledger scan. Polars has figured out — entirely on its own — that only 3 of the 9 available columns are actually needed anywhere in this pipeline, and it will only read those columns from disk. It has also pushed the `debit > 0` filter down to happen *during* the file scan, rather than loading the full dataset first and filtering afterward.

### ! Why This Matters for Real Accounting Data

A university dataset like `general_ledger_sample.csv` has 20 rows — the difference between eager and lazy evaluation is invisible here. But a real company's general ledger for a single fiscal year can easily contain **millions of transaction lines** across dozens of columns. In that setting:

- Reading only the 3 needed columns instead of all 9 can cut memory use dramatically
- Filtering before a join (rather than joining everything, then filtering) avoids doing expensive join work on rows you're about to discard anyway
- These optimizations happen **automatically** once you write your pipeline in lazy form — you don't have to manually reorder your steps for performance

This is precisely the kind of workload common in audit analytics (Chapter 8) and fraud detection (Chapter 9), where you may need to scan an entire year of transactions rather than a sample. Lazy evaluation is one reason polars is gaining adoption for exactly this kind of large-scale accounting analytics work.

### A Note on Pandas and Scale

Pandas itself doesn't have a lazy evaluation mode. For very large datasets in a pandas-based workflow, teams sometimes turn to libraries like **Dask** or **Modin**, which mimic the pandas API while distributing computation — these are beyond the scope of this book, but worth knowing the names of if you encounter a pandas dataset too large to fit comfortably in memory.

## 4.46. Chapter Summary

- pandas and polars share the same core wrangling concepts — filtering, sorting, cleaning, joining, reshaping, grouping — but express them with different syntax; pandas remains the current industry standard, while polars is a fast-growing alternative especially suited to large datasets.
- Cleaning real accounting data typically involves fixing currency formatting, standardizing inconsistent text, parsing unreliable dates, filling missing values, and checking for duplicates — and automated cleaning steps (like `.title()`) still need a human sanity check.
- Inner joins silently drop unmatched rows; left joins preserve every row from your primary table and reveal mismatches — prefer left joins when enriching transaction data with a reference table like a chart of accounts.
- Reshaping between wide (human-readable) and long/tidy (analysis-ready) formats is a routine step, supported in both libraries via pivot/melt (pandas) and pivot/unpivot (polars).
- Grouping and aggregating transaction-level data lets you rebuild summary views — like a trial balance — directly from the general ledger, and is a natural way to sanity-check that debits equal credits.
- Polars' lazy evaluation (`pl.scan_csv() + .collect()`) lets the library optimize an entire pipeline before running it — reading only needed columns and pushing filters earlier — which becomes increasingly valuable as accounting datasets grow into the millions of rows.

## 4.47. Discussion Questions

1. Why might an inner join be the *wrong* default choice when enriching general ledger transactions with chart-of-accounts data, even though it produces a “cleaner-looking” result with fewer rows to review?
2. The `.title()` cleaning step turned "HR" into "Hr". Can you think of another accounting example (an account name, a department, an abbreviation) where automatic text cleaning could introduce a *new* error while fixing another?

3. In your own words, explain why lazy evaluation matters more for a company with millions of general ledger transactions than for the 20-row sample used in this chapter.

## 4.48. Exercises

1. Load `general_ledger_sample.csv` in both pandas and polars, and confirm both libraries produce the same total debits and total credits when grouped by account.
2. Using `messy_general_ledger.csv`, complete the full cleaning workflow (currency formatting, text standardization, dates, missing values, duplicates) in whichever library you're more comfortable with, then repeat it in the other library.
3. Using `chart_of_accounts.csv` and `general_ledger_sample.csv`, perform a left join and identify every general ledger account with no chart-of-accounts match. Write one sentence describing what you'd do next if this were a real client engagement.
4. Rewrite the eager pipeline in the Lazy Frames section as your own lazy pipeline, but group by department instead of `account_type`. Run `.explain()` on your version and describe, in your own words, what optimization(s) you can identify in the output.



## **5. Exploratory Data Analysis for Financial Data**



## **6. Data Visualization for Accounting**





**Part III.**

**Part III: Applied Accounting  
Analytics**



## **7. Financial Statement Analysis with Python**



## 8. Audit Analytics



## **9. Fraud Detection and Anomaly Detection**





## **10. Revenue Recognition and Transaction-Level Analysis**



## 11. Tax Analytics



**Part IV.**

**Part IV: Predictive Methods and  
GenAI**



## **12. Introduction to Predictive Modeling**





## **13. Generative AI in Accounting**



**Part V.**

**Part V: Communication and  
Capstone**



## 14. Automated Reporting and Dashboards with Quarto



## **15. Data Ethics, Governance, and Reproducibility**





## 16. Capstone Project



# References

Tim Stobierski. Data Wrangling: What It Is & Why It's Important, 2021. URL <https://online.hbs.edu/blog/post/data-wrangling>.



## **A. A Primer on Python**



## **B. Financial Statement Basics**

